Module 3: Processing Text for NLP

AWS Academy Natural Language Processing

Module overview

Sections

- Text processing overview
- Getting text
- Text preprocessing
- Vectorizing text
- Advanced processing
- Storing and visualizing unstructured data

Labs

- Guided Lab: Extracting Text from Webpages and Images
- Guided Lab: Processing Text
- Guided Lab: Encoding and Vectorizing Text

Module objectives

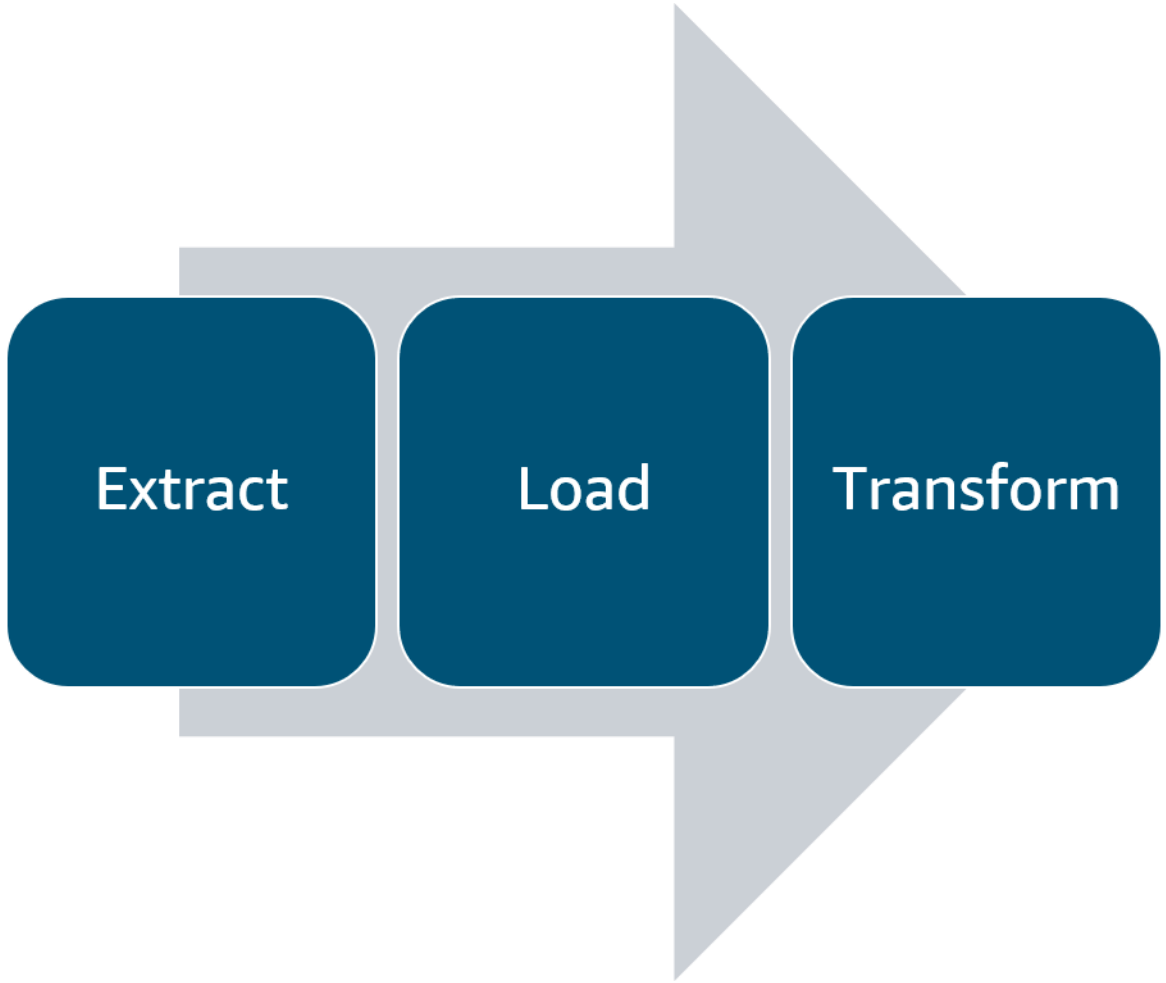
- At the end of this module, you should be able to:
- Define the processing steps for text
- Implement text extraction to obtain data from webpages
- Identify tokenizers for text processing
- Describe the techniques and tasks used in processing text
- List the advanced processing steps for natural language processing (NLP)
- Identify the storage services for text in Amazon Web Services (AWS)

Section 1: Text processing overview

Module 3: Processing Text for NLP

Text processing stages

- Extract text from data sources
  - Web scraping
  - Document conversion
  - Automation
- Load into a data store
  - Code
  - AWS Glue
  - Amazon Data Wrangler
- Transform, feature-engineer
  - Process
  - Tokenize

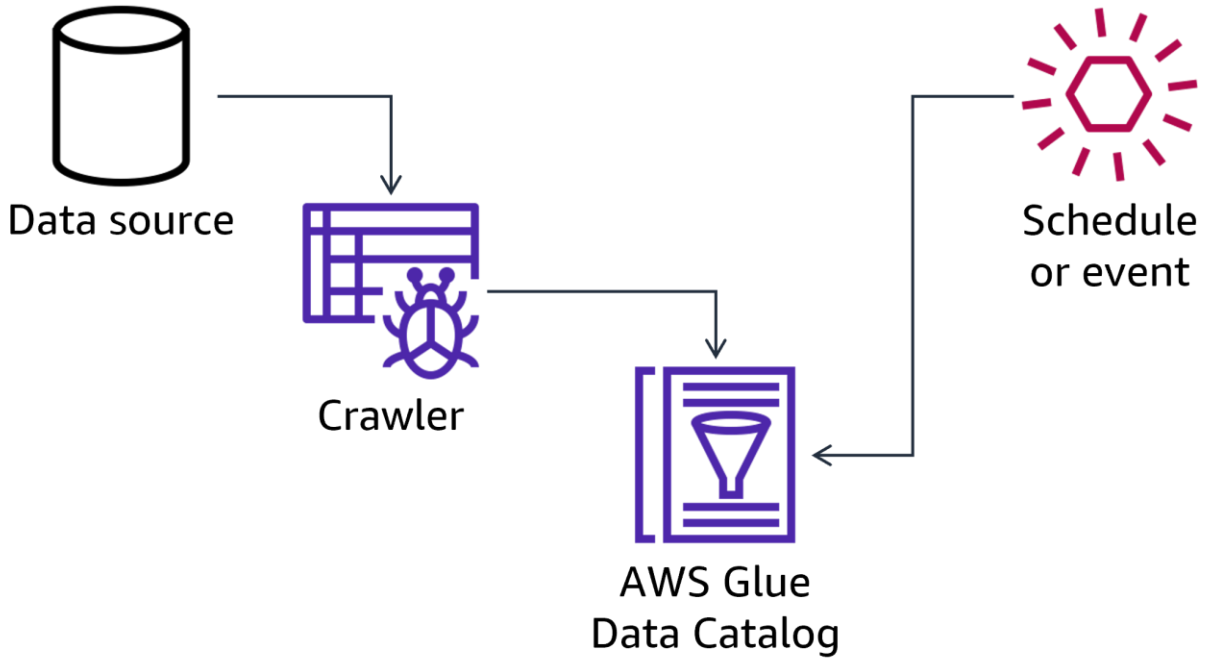


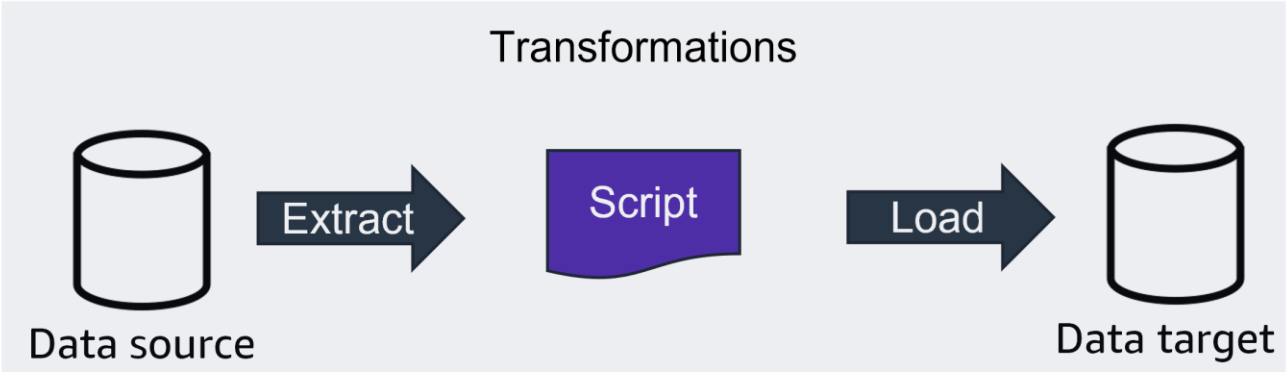
Types of text extraction

- Unstructured Data
  - Raw text
    - Extract words or sentences
  - Image data
    - Extract text from images
- Structured data
  - Tabular data, such as banking records or employment records
  - Read data and structure from forms

AWS Glue

- Provides automation for your extract, transform, and load (ETL) process
  - Crawler discovers metadata for your data sources
  - Metadata stored in a Data Catalog
  - Transform scripts can be created by AWS Glue or a user
- Transform scripts can be run on demand, or scheduled

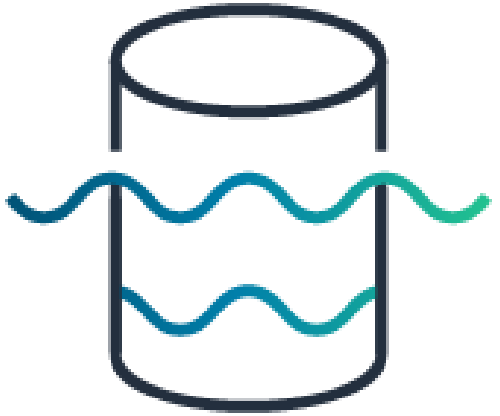




AWS Glue use cases

DevOps pipeline

Threat detection



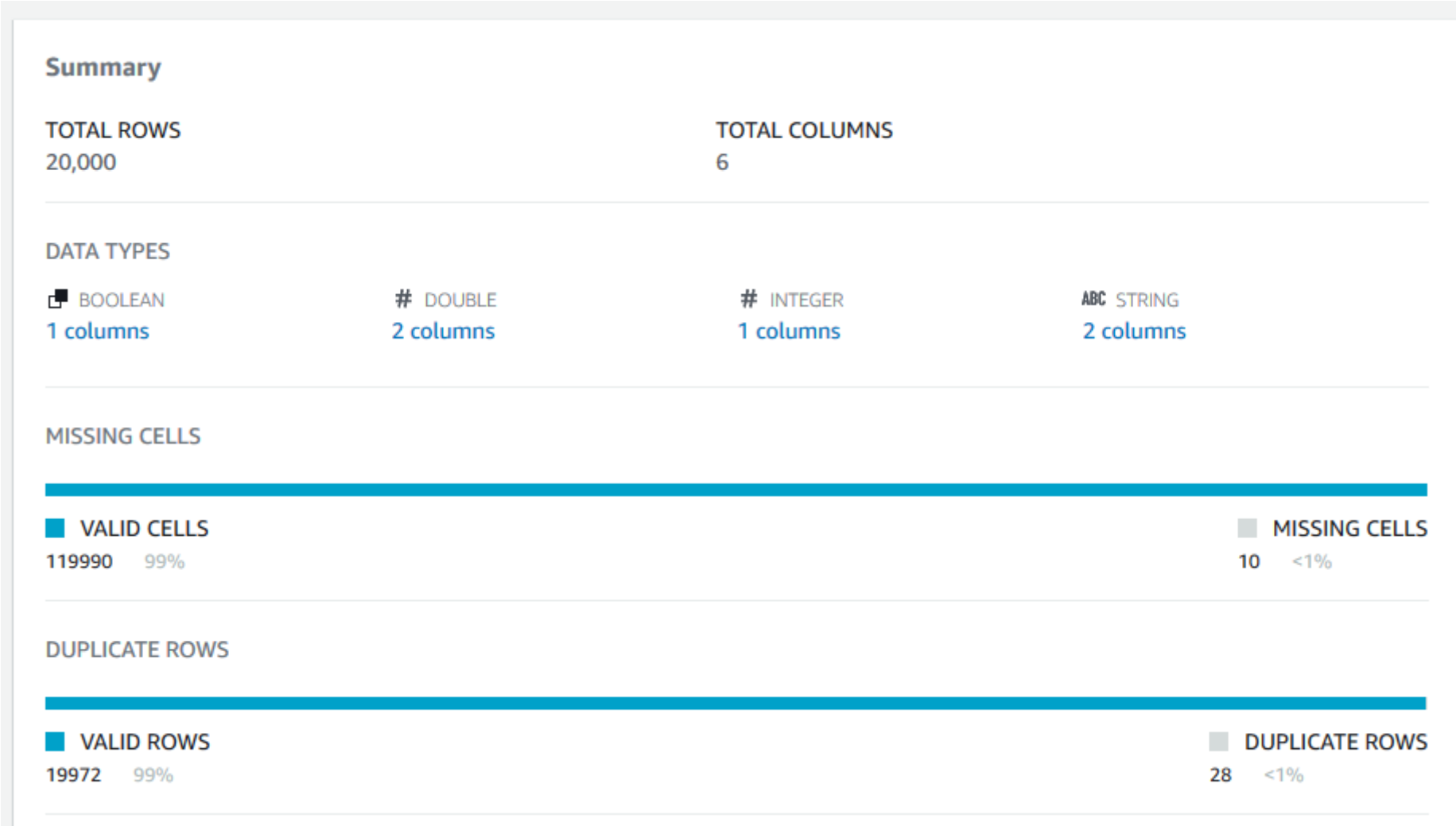
Data lake extraction

Migration of older data

AWS Glue DataBrew

- Visually explore data
- Profile data to highlight patterns
- Detect anomalies
- Clean and normalize data without writing code
- Save and reuse transformations on new data

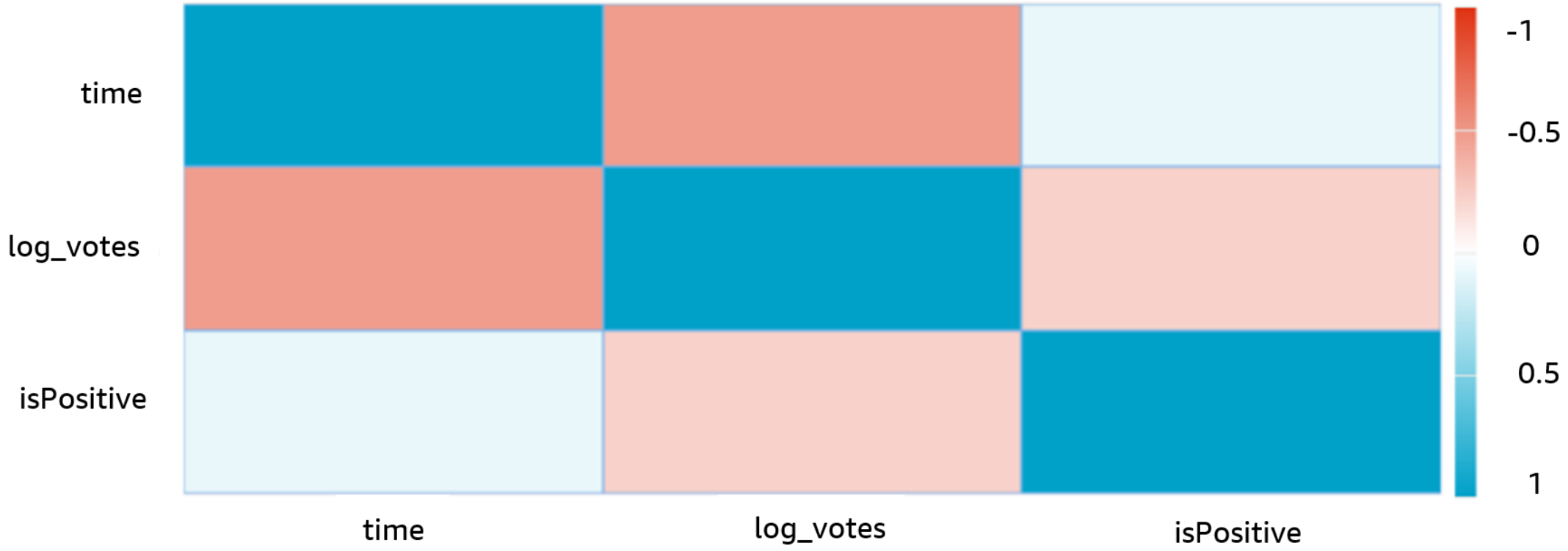
AWS Glue DataBrew: Data Profile Summary



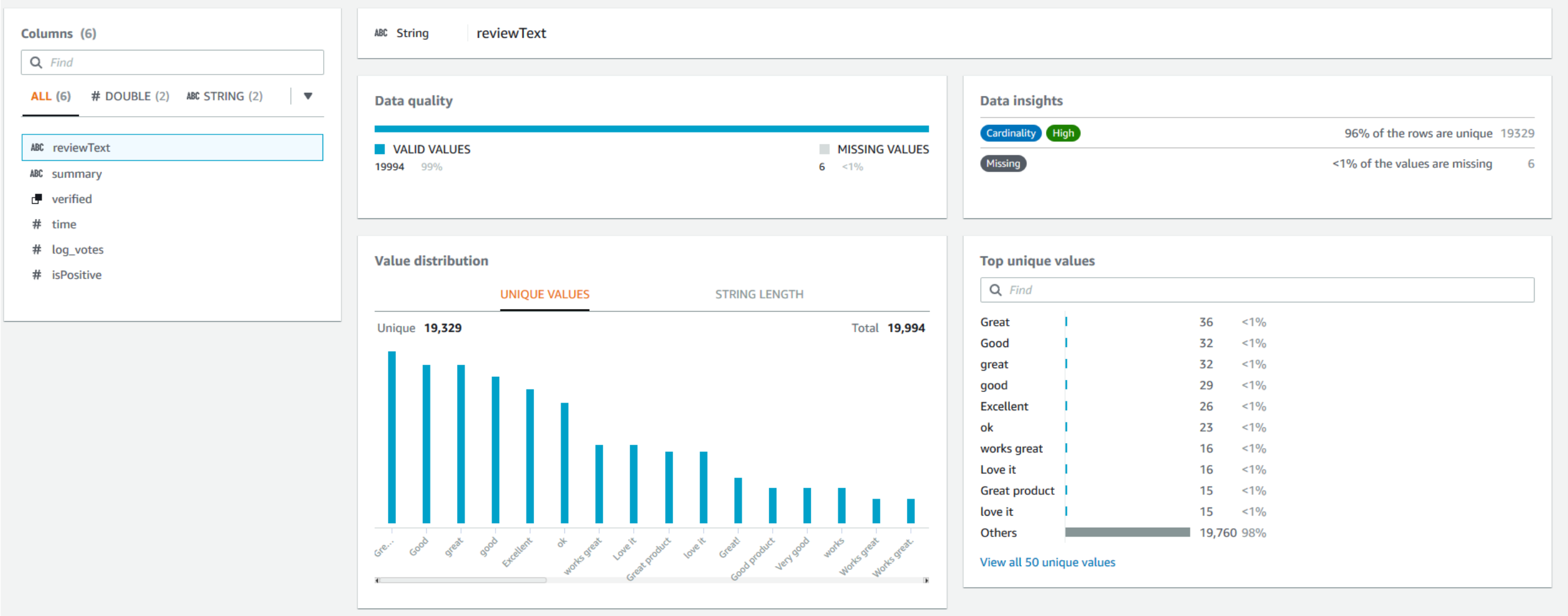
AWS Glue DataBrew: Data Correlations

Correlations

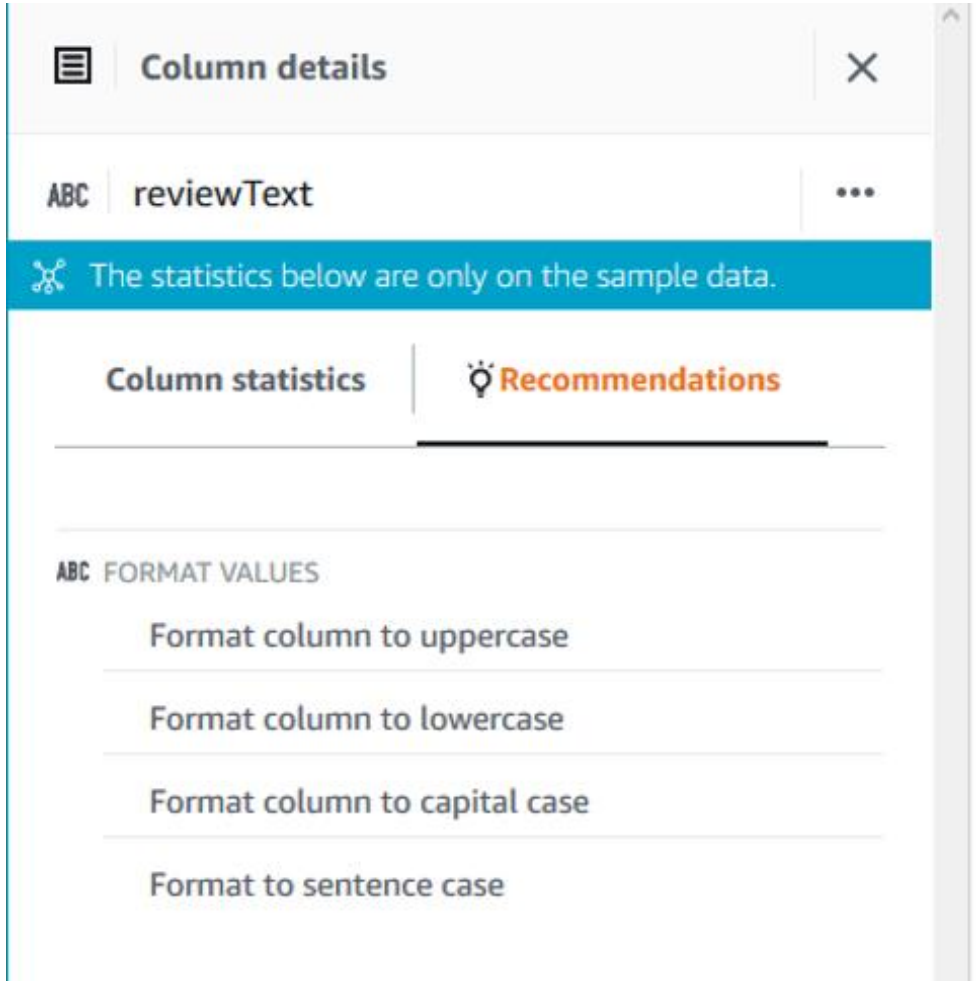
Correlation coefficient (r) defines how closely two variables are related. It ranges from -1.0 to +1.0, where 0 means there is no relationship between the variables.



AWS Glue DataBrew: Column statistics



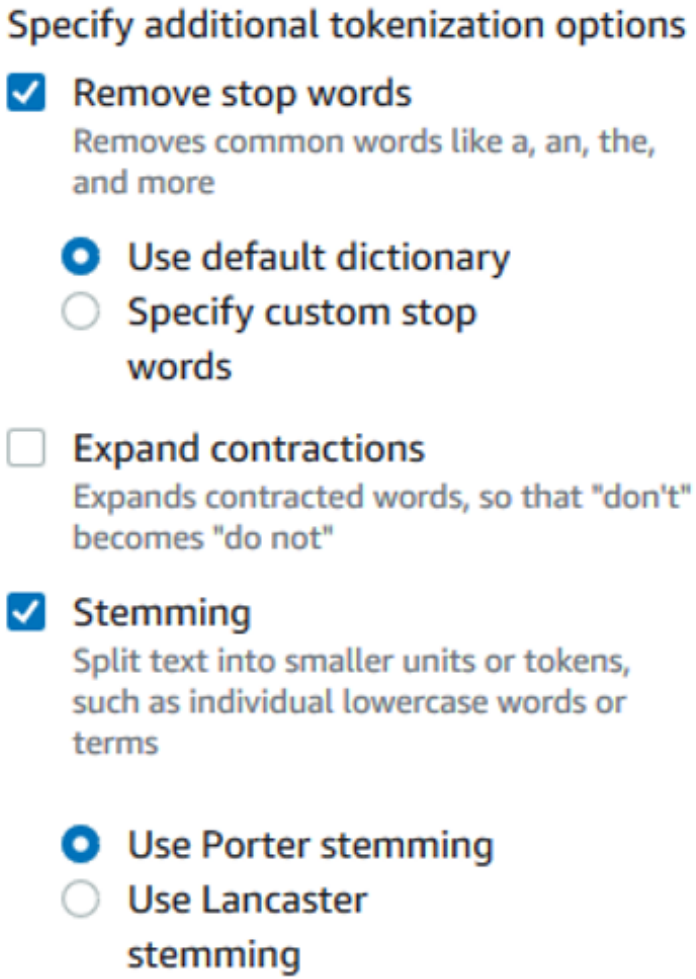
AWS Glue DataBrew: Projects



- Column details
  - Statistics
  - Recommendations
- Predefined column actions:
  - Format
  - Clean
  - Extract
  - Missing values
  - Word tokenization
  - Categorical mapping

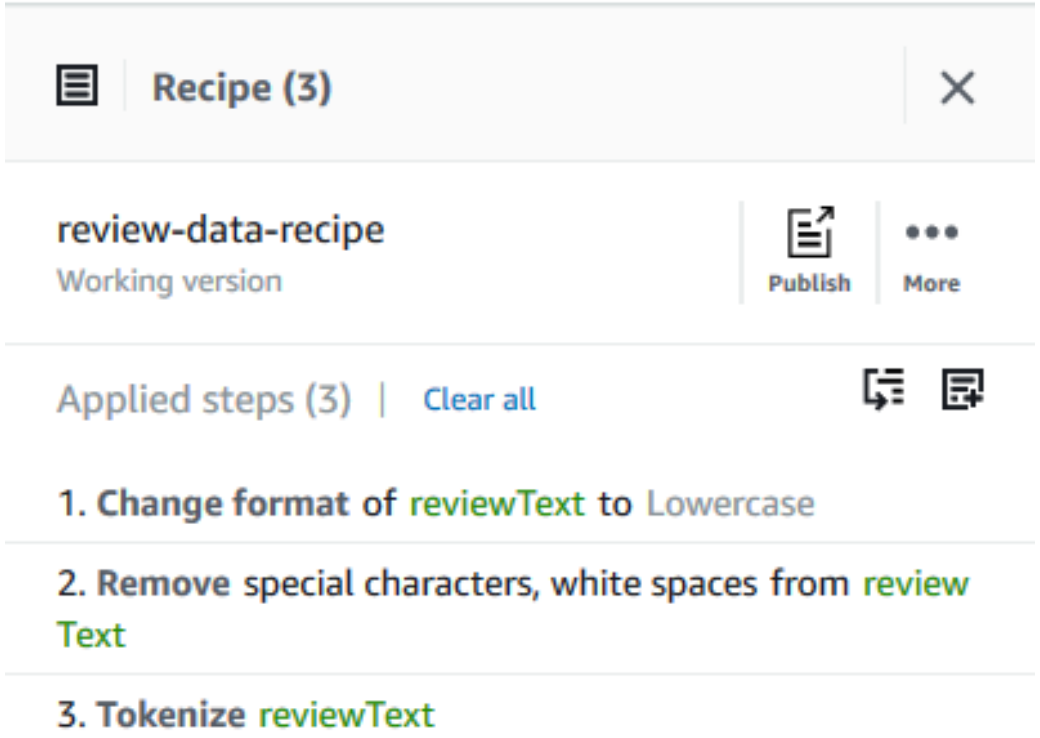
AWS Glue DataBrew: Tokenization

- Tokenization
  - Stopword removal
  - Expand contractions
  - Stemming



AWS Glue DataBrew: Recipes

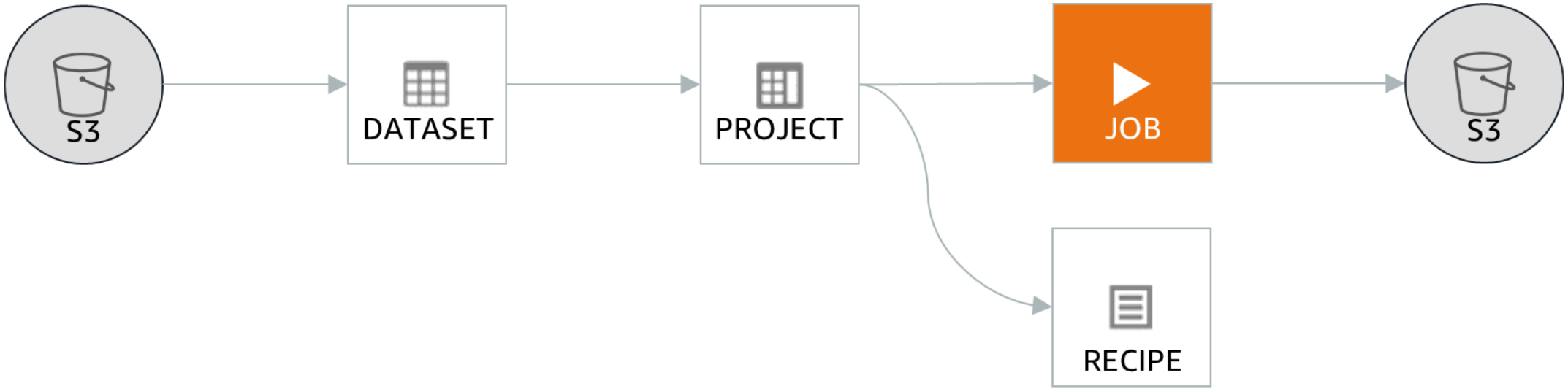
Recipes are a list of steps



AWS Glue DataBrew: Recipe jobs

Recipe jobs

- Project = dataset + recipe
- Schedule
- Data lineage

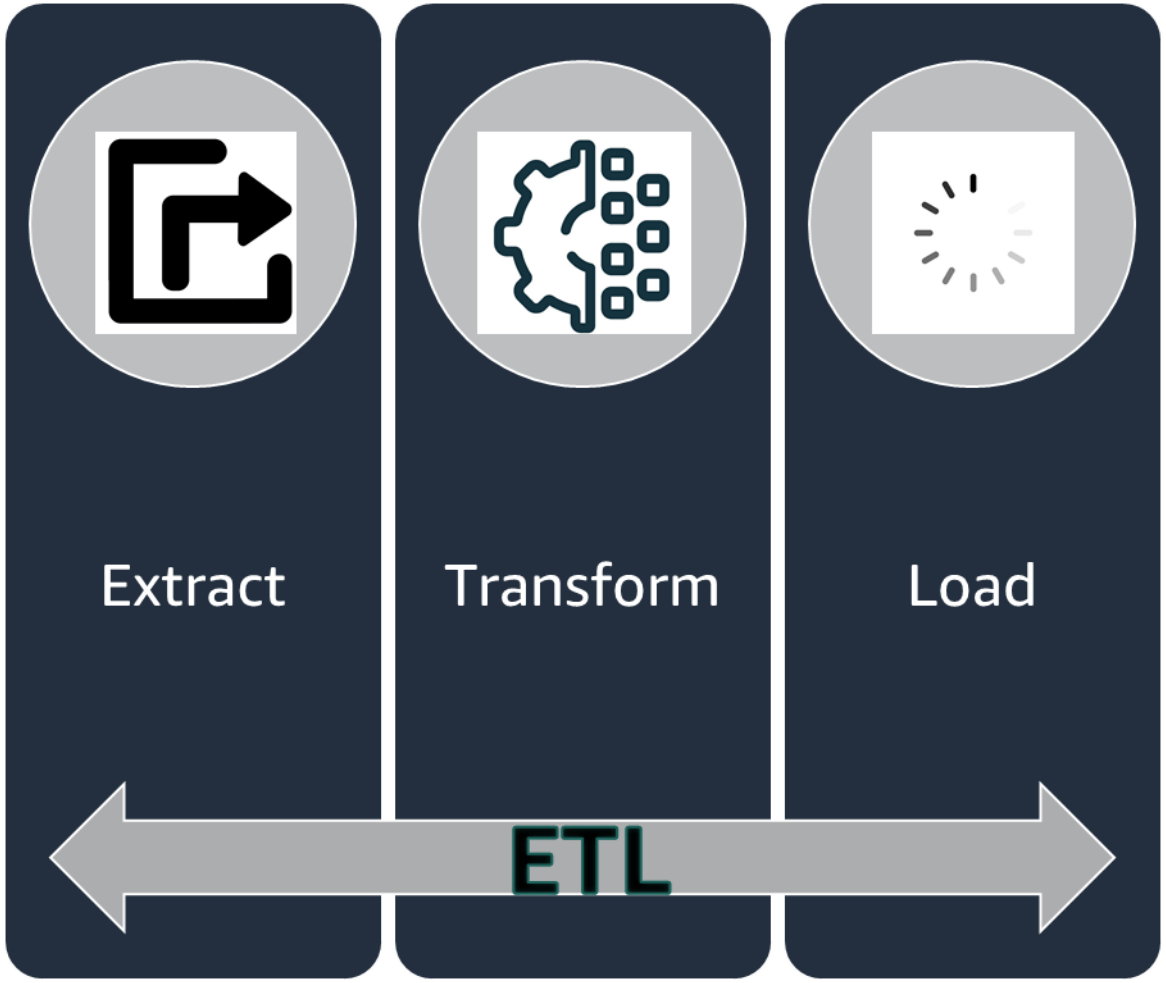


Amazon SageMaker Data Wrangler

- Is a part of Amazon SageMaker Studio
- Create data-preparation and feature-engineering flows
  - Import data from Amazon S3, Amazon Athena, or Amazon Redshift
  - Create a data flow to complete data preparation steps
  - Estimate machine learning (ML) model accuracy
  - Transform data, including vectorization
  - Build visualizations
  - Export to notebook, Python script, or SageMaker pipeline

Section 1 summary

- Getting text is the first step in the ETL process
- Web scraping and extracting from data stores are two common ways to get text
- Tools
  - Amazon Textract
  - Python libraries
  - AWS Glue
  - AWS Glue DataBrew
  - Amazon SageMaker Data Wrangler

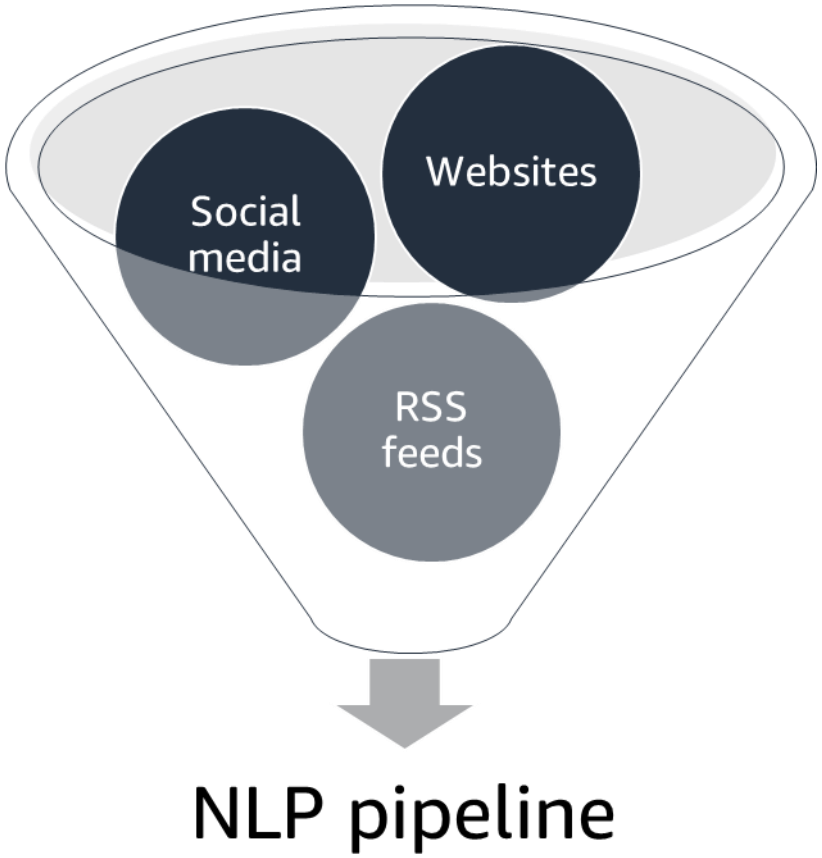


Section 2: Getting text

Module 3: Processing Text for NLP

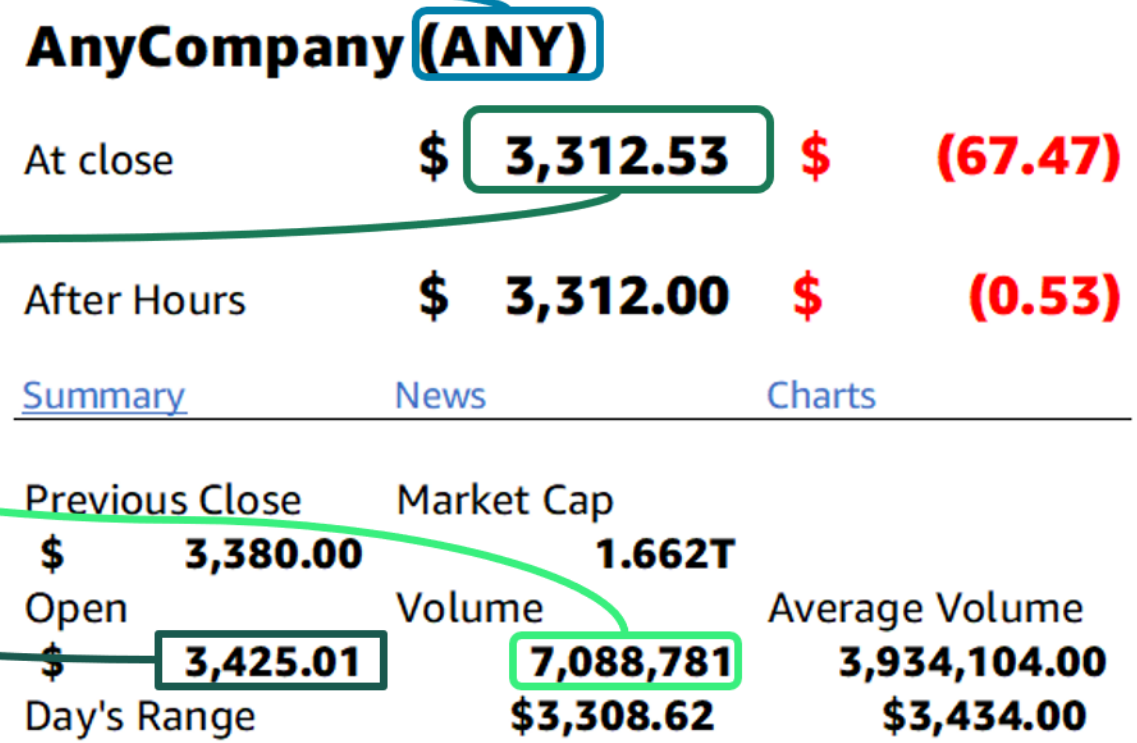
Extracting text from the web

- You can automate data capture for your NLP application.
- Example use cases –
  - Marketing analysis: Gather product information.
  - Hiring: Harvesting resumes.
  - News analysis: Twitter feeds.
- Python libraries –
  - Beautiful Soup: Text-parsing tools for Hypertext Markup Language (HTML) and Extensible Markup Language (XML).
  - Scrapy: A set of tools for more complex projects.



Web scraping example

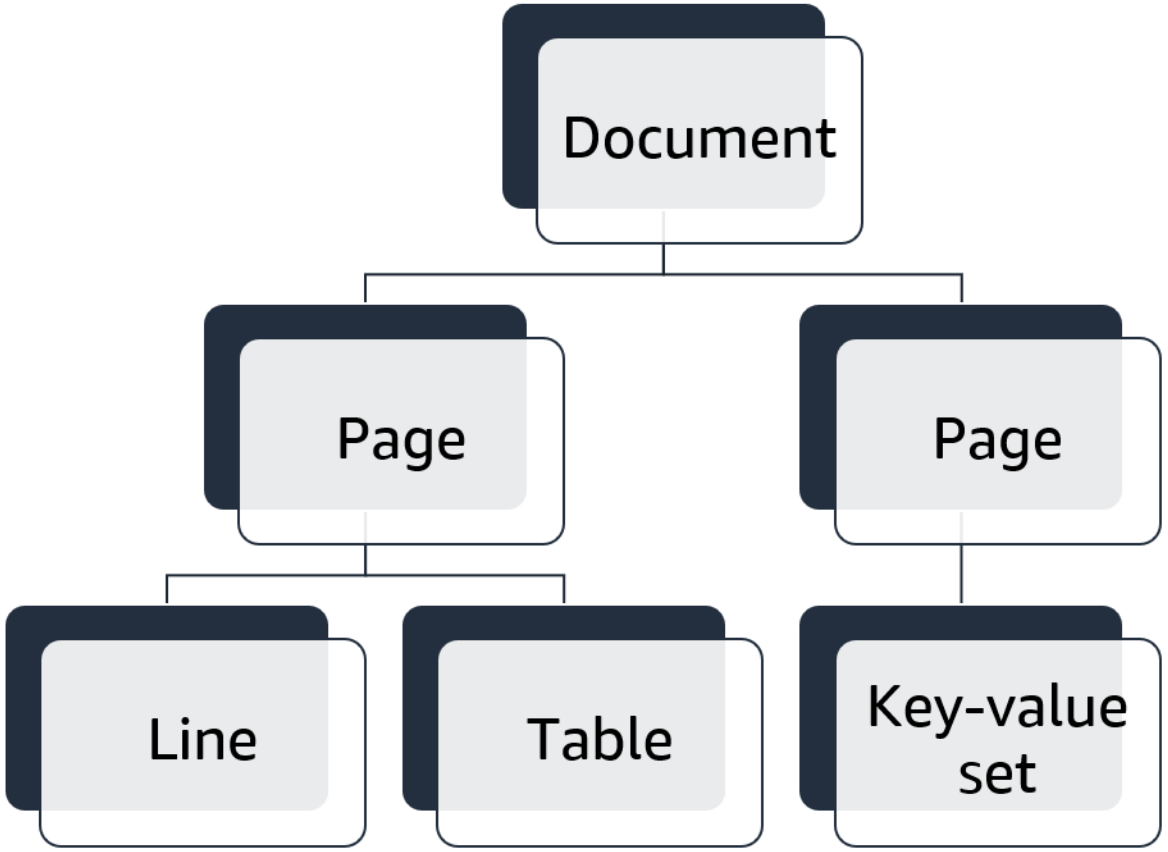
StockCode	Open	Close	Volume
ANY	3,425.01	3,312.53	7,088,781



Amazon Textract

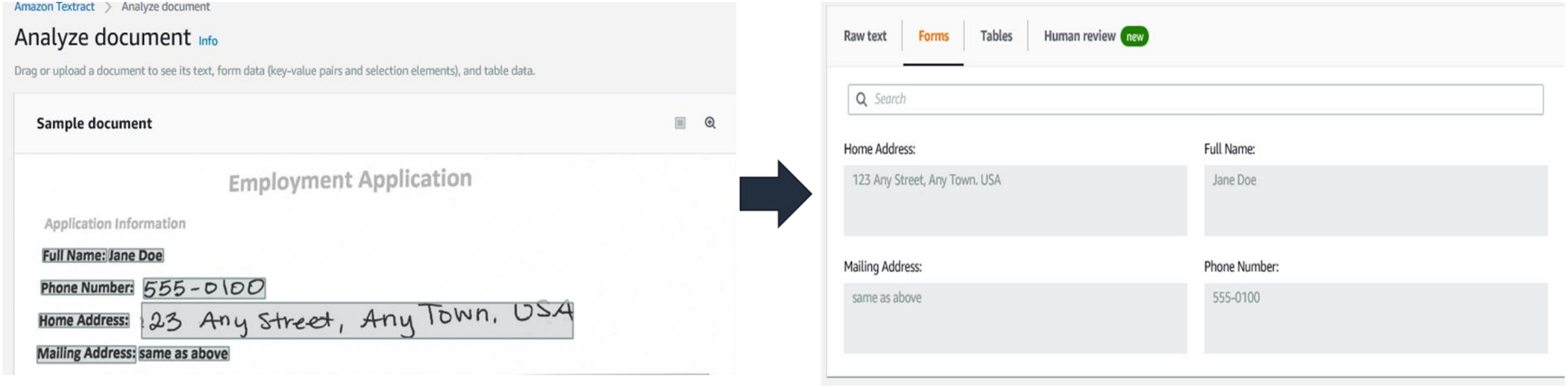
- Retrieves data directly out of scanned documents
- Extracts data from:
  - Pages of text
  - Form data
    - Retrieved as key-value pairs
  - Tables of data
    - Retrieved as columnar data
  - Selection elements
    - For example, check boxes or option buttons
    - Retrieved from forms and tables

Amazon Textract document structure



Load text data into a processing pipeline

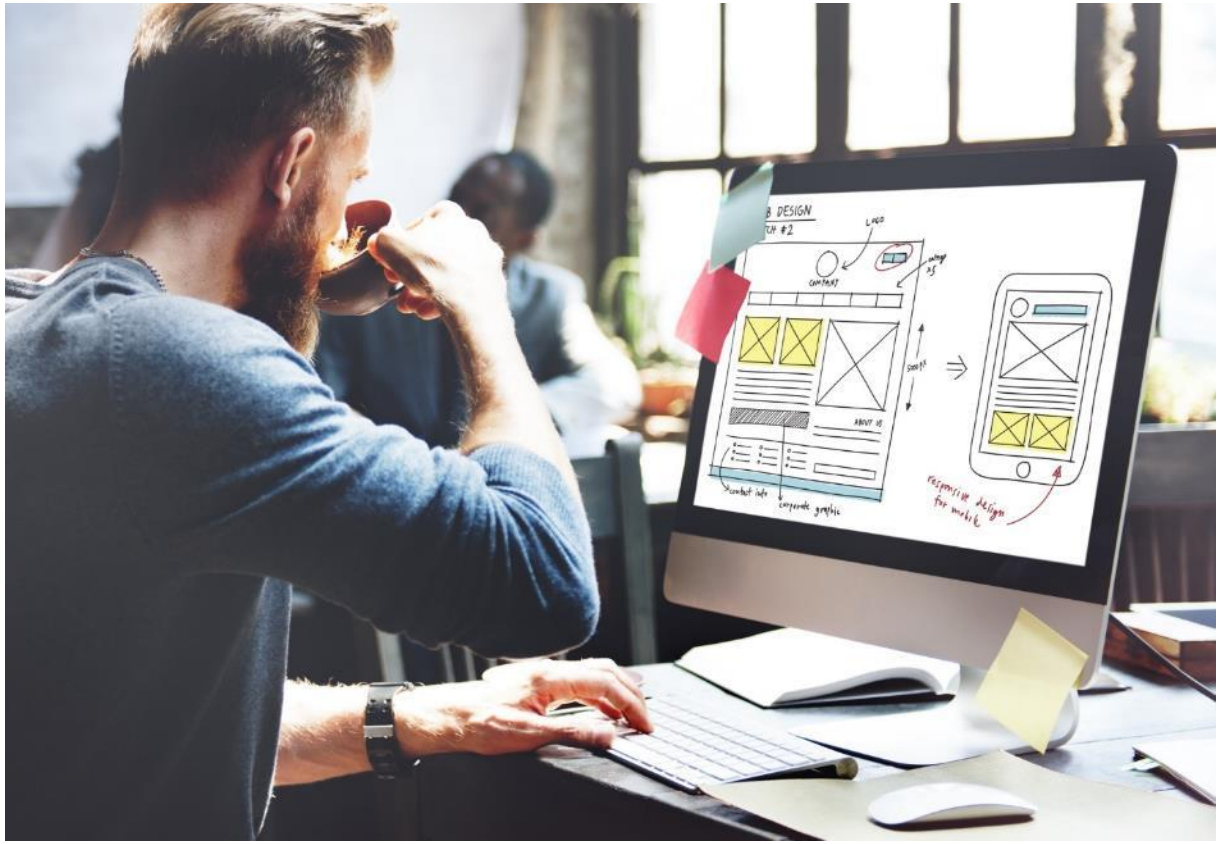
- Amazon Textract: Can automate text extraction with the AWS Command Line Interface (AWS CLI) or application programming interface (API).



Section 2 summary

- Text extractions tools:
  - Beautiful Soup, Scrapy, Amazon Textract
- Amazon Textract
  - Extracts text, tabular data, and forms
  - Extracts text from data images

Module 3 Guided Lab 1: Extracting Text from Webpages and Images



Section 3: Text preprocessing

Module 3: Processing Text for NLP

Text cleanup tasks

- Remove leading white space and trailing white space
- Changing case
- Removing HTML tags
- Converting numeric values to text representations
- Removing punctuation
- Reformatting text

Tokenization

- A process that splits text and document into small parts by using white space and punctuation.
- Example:

Sentence	Tokens
“I don’t like eggs.”	“I”, “don”, “t”, “like”,“eggs”, “.”

- These tokens will be used in the next steps in the pipeline.

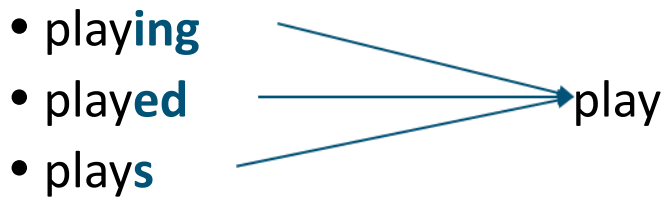
Stopword removal

- **Stopwords:** Some words that **frequently appear** in texts, but they **don't contribute much to the overall meaning**
  - Common stopwords: *a*, *the*, *so*, *is*, *it*, *at*, *in*, *this*, *there*, *that*, *my*
- **Example**:

Original sentence	Without stopwords
"There is a tree near the house"	"tree near house"

Stemming

- Set of rules to slice a string to a substring
  - Substring usually refers to a more general meaning
  - Goal is to remove word affixes (particularly suffixes), such as *-s*, *-es*, *-ing*, *-ed*



- One issue – Stemming doesn't usually work with irregular forms, such as irregular verbs
  - Examples: Taught, brought

Lemmatization

- Is similar to stemming, but more advanced
- Uses a lookup dictionary
  - Handles more situations and usually works better than stemming to map a word back to its stem form
  - For best results, provide the correct word position tags: adjective, noun, verb

Stemming versus lemmatization

As mentioned previously, lemmatization is a more complex method and it usually works better than stemming to map a word to its stem form.

Consider the following example.

Original sentence

"the children are playing outside. the weather was better yesterday."

- Stemming =>
  “the children are **play** outside. the weather was better yesterday”
- Lemmatization =>
  “the child **be play** outside. the weather **be good** yesterday”

Text processing with managed services

AWS Glue DataBrew

- Text cleanup
- Stopword removal
- Tokenization
- Stemming

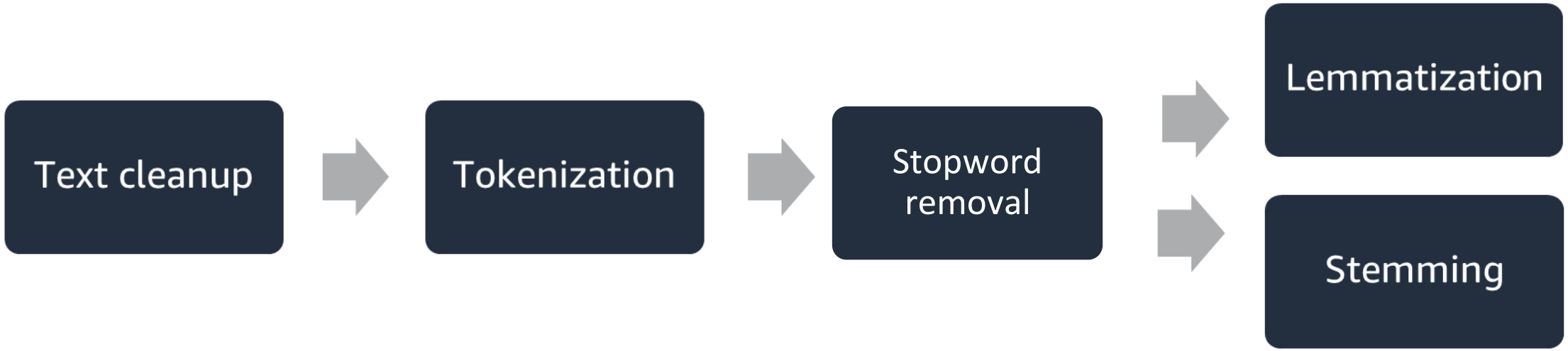
Amazon SageMaker Data Wrangler

- Text cleanup
- Tokenization
- Stemming
- Vectorization
- Custom scripts

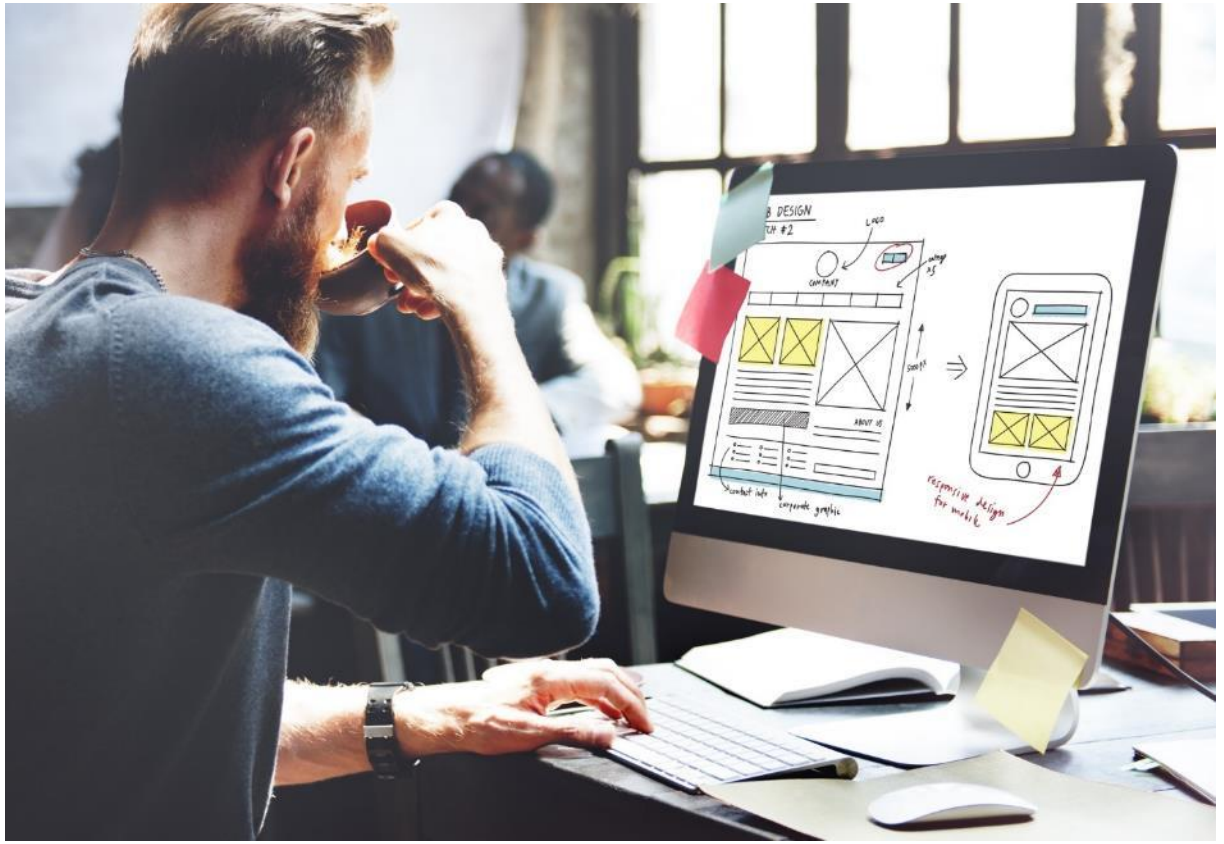
Section 3 summary

Key preprocessing tasks:

- Text cleanup
- Tokenization
- Stopword removal
- Stemming
- Lemmatization



Module 3 Guided Lab 2: Processing Text



Section 4: Vectorizing text

Module 3: Processing Text for NLP

BOW

- The bag-of-words (BOW) method converts text data into numbers.
- BOW does this conversion by –
  - Creating a vocabulary from the words in all documents.
  - Calculating the occurrences of words:
    - Binary (present or not)
    - Word counts
    - Frequencies
- OOV – Out-of-vocabulary problem

BOW: Binary

- Simple example that uses a binary flag:

	a	cat	dog	is	it	my	not	old	wolf
“It is a dog.”	1	0	1	1	1	0	0	0	0
“my cat is old”	0	1	0	1	0	1	0	1	0
“It is not a dog, it is a wolf.”	1	0	1	1	1	0	1	0	1

BOW: Word counts

- Simple example that uses word counts:

	a	cat	dog	is	it	my	not	old	wolf
“It is a dog.”	1	0	1	1	1	0	0	0	0
“my cat is old”	0	1	0	1	0	1	0	1	0
“It is not a dog, it is a wolf.”	2	0	1	2	2	0	1	0	1

TF

Term frequency (TF): *Increases* the weight for *common* words in a ***document***

$$tf(term, doc) = \frac{number\ of\ times\ the\ term\ occurs\ in\ the\ doc}{total\ number\ of\ terms\ in\ the\ doc}$$

	a	cat	dog	is	it	my	not	old	wolf
“It is a dog.”	0.25	0	0.25	0.25	0.25	0	0	0	0
“my cat is old”	0	0.25	0	0.25	0	0.25	0	0.25	0
“It is not a dog, it is a wolf.”	0.22	0	0.11	0.22	0.22	0	0.11	0	0.11

IDF

term	idf
a	log(3/3)+1=**1**
cat	log(3/2)+1=**1.18**
dog	log(3/3)+1=**1**
is	log(3/4)+1=**0.87**
it	log(3/3)+1=**1**
my	log(3/2)+1=**1.18**
not	log(3/2)+1=**1.18**
old	log(3/2)+1=**1.18**
wolf	log(3/2)+1=**1.18**

Inverse document frequency (IDF): *Decreases* the weights for *commonly* used words, and *increases* weights for *rare* words that are in the *vocabulary*

$$idf(term) = \log\left(\frac{n_{documents}}{n_{documents\ containing\ the\ term} + 1}\right) + 1$$

$$example\colon\ \ idf(\text{“}cat\text{”}) = 1.18$$

TF-IDF

Term frequency-inverse document frequency (TF-IDF): Combines term frequency and inverse document frequency.

$$tf_{idf}(term, doc) = tf(term, doc) * idf(term)$$

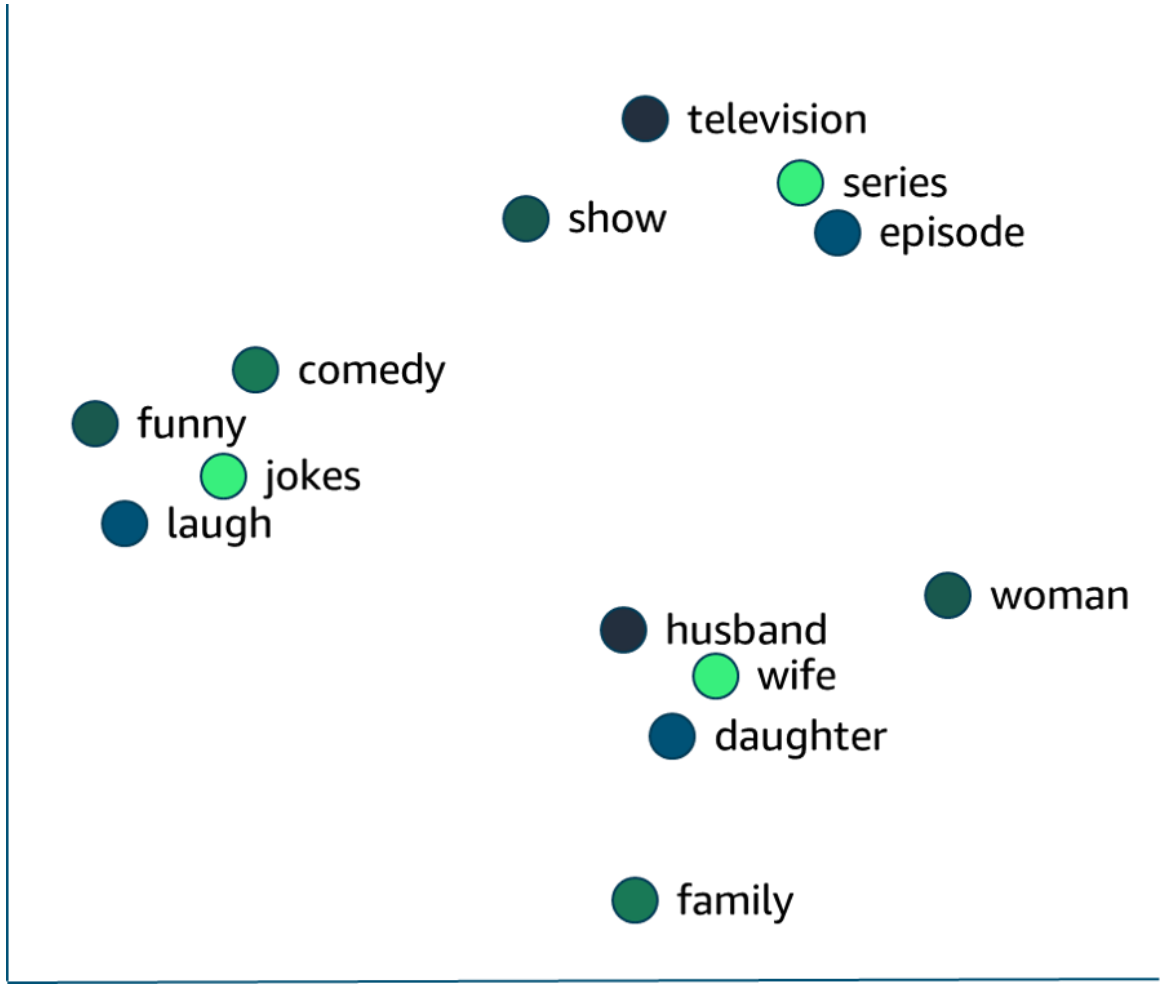
	a	cat	dog	is	it	my	not	old	wolf
“It is a dog.”	0.25	0	0.25	0.22	0.25	0	0	0	0
“my cat is old”	0	0.3	0	0.22	0	0.3	0	0.3	0
“It is not a dog, it is a wolf.”	0.22	0	0.11	0.19	0.22	0	0.13	0	0.13

N-gram

- An n-gram is a sequence of *n* tokens from a given sample of text or speech.
- You can include n-grams in your term frequencies.

Sentence	1-gram (uni-gram):	2-gram (bi-gram):
It is not a dog, it is a wolf	“it”, “is”, “not”, “a”, “dog”, “it”, “is”, “a”, “wolf”	“it is”, “is not”, “not a”, “a dog”, “dog it”, “it is”, “is a”, “a wolf”

Word vectors: Word2Vec



- It learns an embedding vector for each word.
- It produces hundreds of dimensions.
- Words with similar semantics are grouped close together.
- You can use pretrained models.
- Vectors can be visualized by using t-distributed stochastic neighbor embedding (t-SNE).
- Words that are not in corpus return zeroes.
- Vectors are used for downstream NLP tasks.

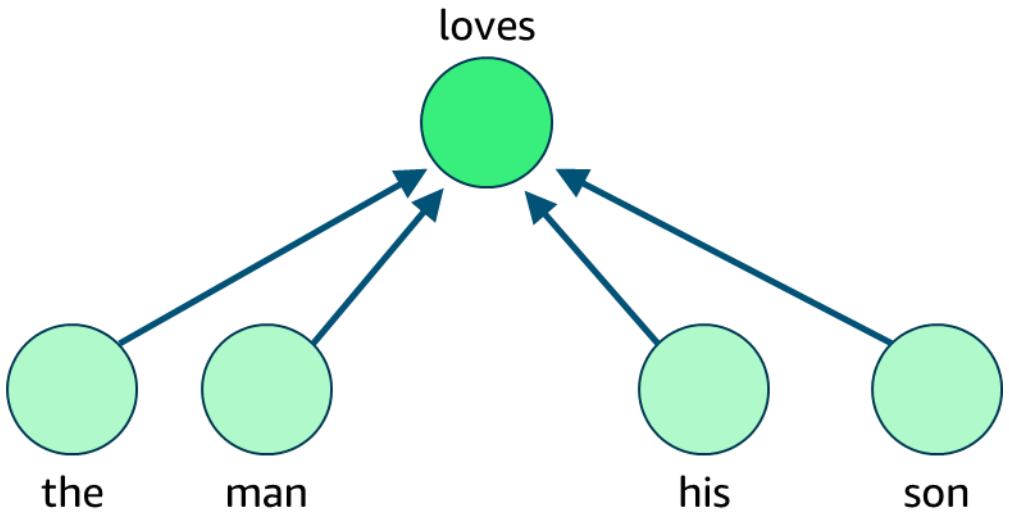
Word2Vec: CBOW and Skip-Gram

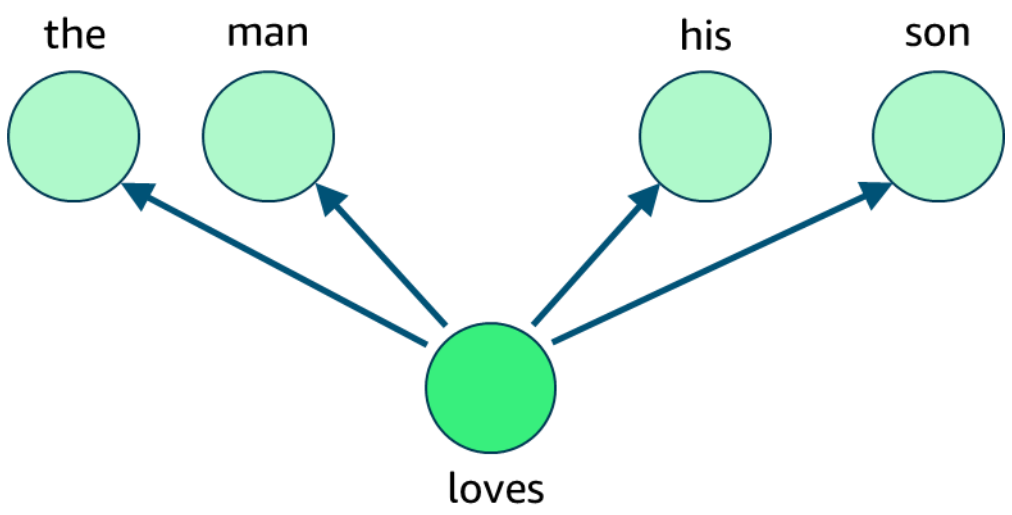
Continuous bag-of-words (CBOW)

Generate center words from context words

Skip-Gram

A word can be used to generate the words that surround it





Amazon SageMaker BlazingText

- Highly optimized implementation of Word2vec
  - Accelerated training on GPUs, uses highly optimized CUDA kernels
- Provides CBOW and Skip-Gram training architectures
- Output (vectors.txt) are compatible with Gensim and spaCy
- Subwords can generate vectors for OOV words
- Text classification

Text vectorization with managed services

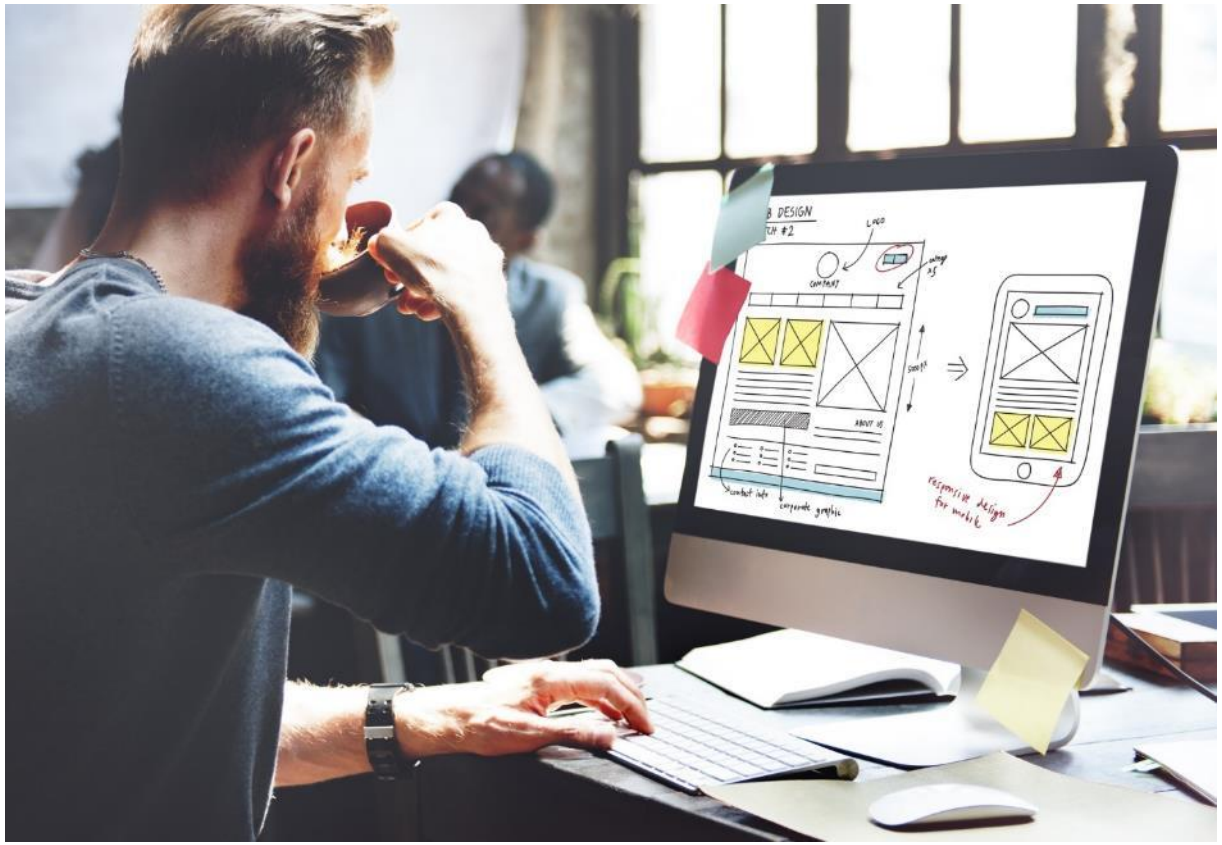
AWS Glue DataBrew

- Not available

Amazon SageMaker Data Wrangler

- Count vectorizer
  - Numeric and binary
- TF-IDF
- Generates columns or a matrix

Module 3 Guided Lab 3: Encoding and Vectorizing Text



Section 4 summary

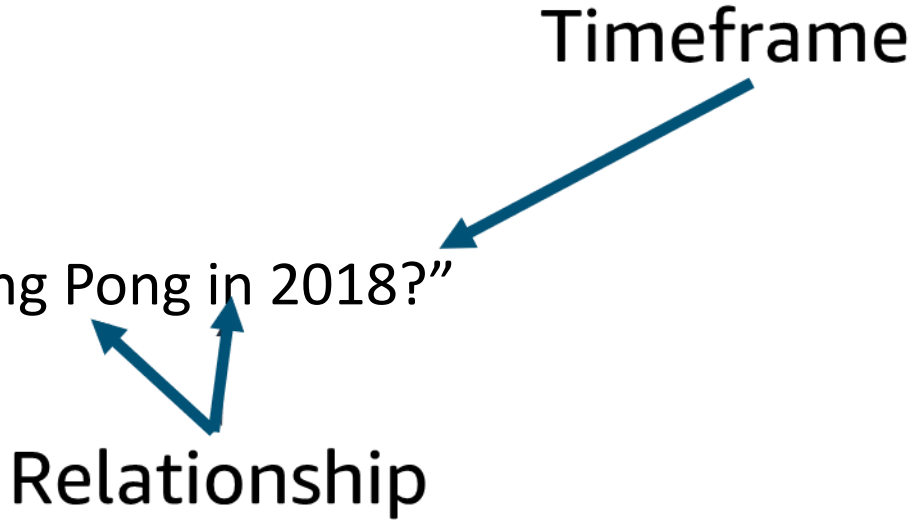
- Bag-of-words (BOW):
- Term frequency (TF)
- Inverse document frequency (IDF)
- Term frequency – inverse document frequency (TF-IDF)
- N-grams
- Word vectors
  - Continuous bag-of-words (CBOW) and Skip-Gram
- BlazingText
- Amazon SageMaker Data Wrangler

Section 5: Advanced processing

Module 3: Processing Text for NLP

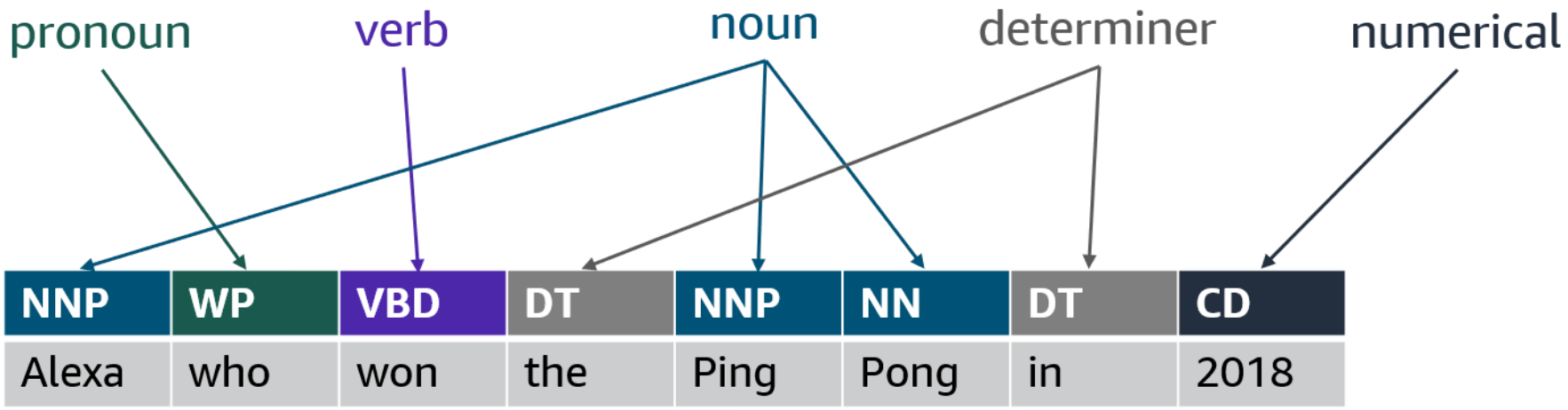
Syntactic analysis and semantic analysis

- Looks at:
  - Word order and meaning
  - Retaining stopwords
  - Morphology of words
  - Parts of speech (POS) in a sentence
- Why?
  - Consider Amazon Alexa
  - “Alexa, who won the Ping Pong in 2018?”



POS tagging

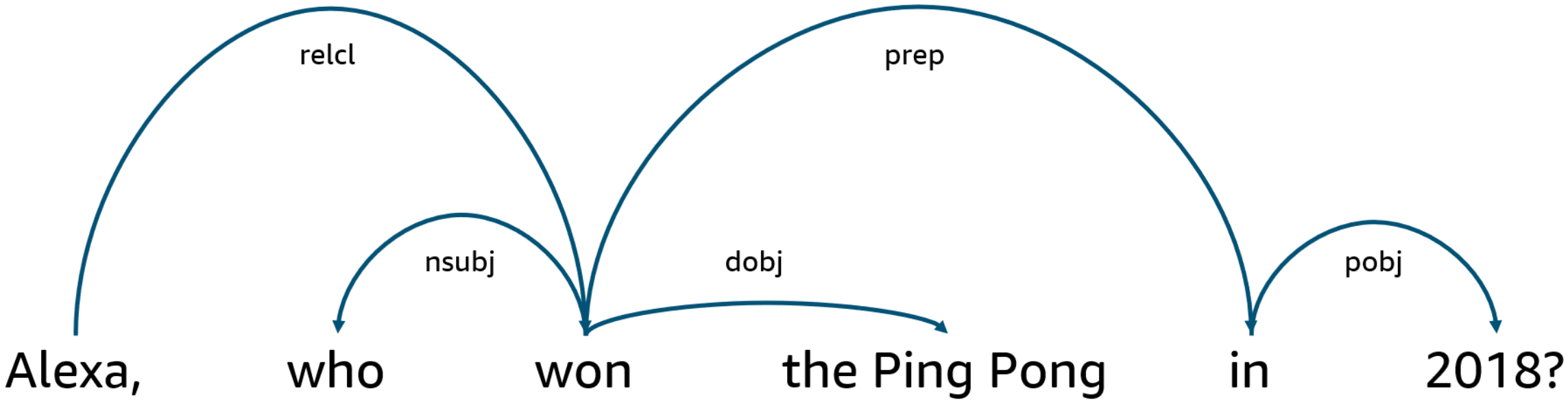
- Part-of-speech (POS) categories
  - Verb, noun, adjective, adverb, pronoun, preposition, conjunction, interjection, numerical, and determiner



Constituency parsing

- Identify common grammatical patterns
- Noun phrases
  - “A packet of chips was included with lunch”
- Verb phrases
  - “I will be going to university in the fall”
- Prepositional phrases
  - “I wanted to live near the beach”

Dependency parsing



relcl	Relative clause modifier
nsubj	Nominal subject
dobj	Direct object
pobj	Object of preposition
prep	Prepositional modifier

Coreference resolution

- Anaphoric – Refer to previous text
  - Diego went for a motorcycle ride. He said it was amazing!
  - He refers to Diego.
  - It refers to the motorcycle ride.
- Cataphoric – Refer to future text
  - Every time she goes for a motorcycle ride, Akua has a smile on her face.
  - she refers to Akua.

Section 5 summary

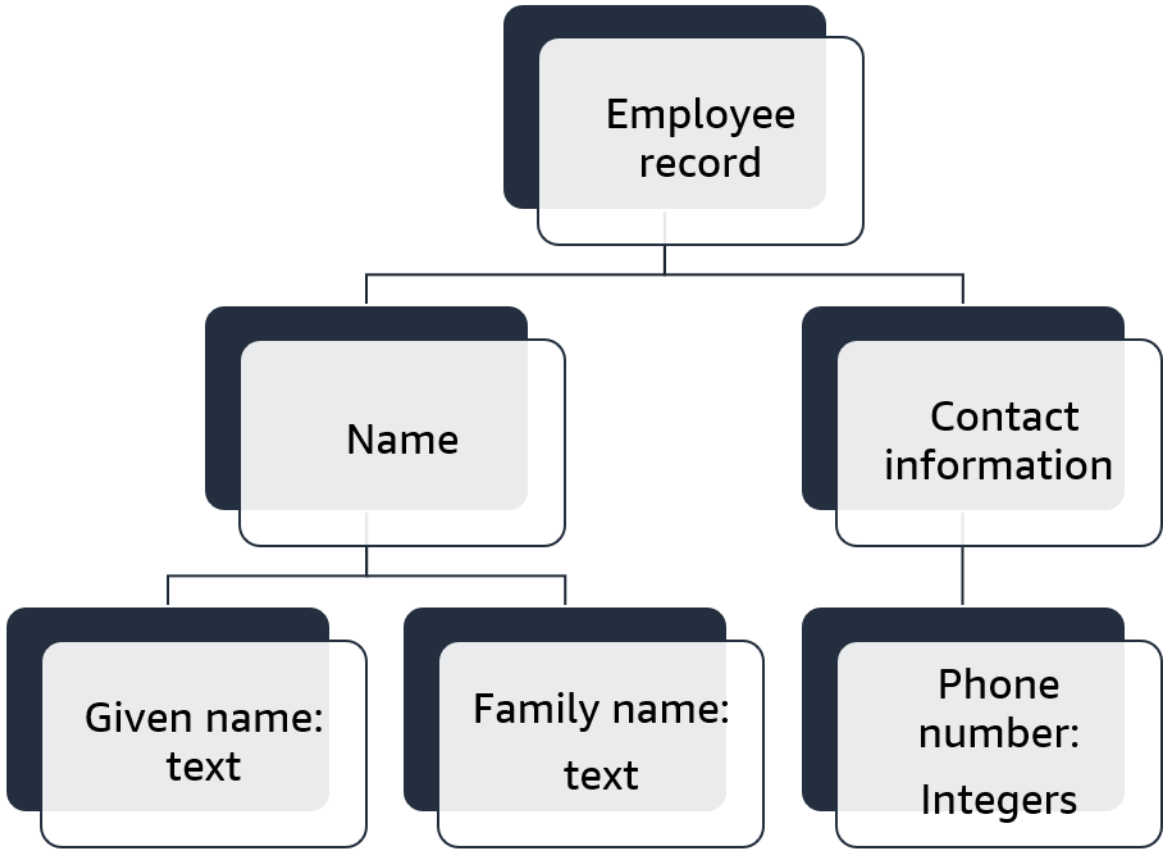
- Understand the meaning of sentences:
  - POS tagging
  - Constituency parsing
  - Dependency parsing
  - Coreference resolution
- Applications:
  - Information retrieval
  - Chatbots
  - Virtual assistants

Section 6: Storing and visualizing unstructured data

Module 3: Processing Text for NLP

Structured versus unstructured data

Structured



Unstructured

Blog post	Title: Day 1 of class
	Date: empty field
	Text: I love NLP…

AWS storage services for unstructured data

Amazon Simple Storage Service (Amazon S3)

- Virtually unlimited object storage
- Most common storage service for machine learning
- Suitable for both structured and unstructured data

Amazon Elasticsearch Service (Amazon ES)

- Managed service for building search-based applications
- Works with Kibana for data visualization
- Automated data loading
- Integrates with language plugins

Amazon Neptune

- Managed graph database service
- Supports common graph models
- Automated backup to Amazon S3
- Provides read replicas for high performance

Use cases for Amazon S3 NLP

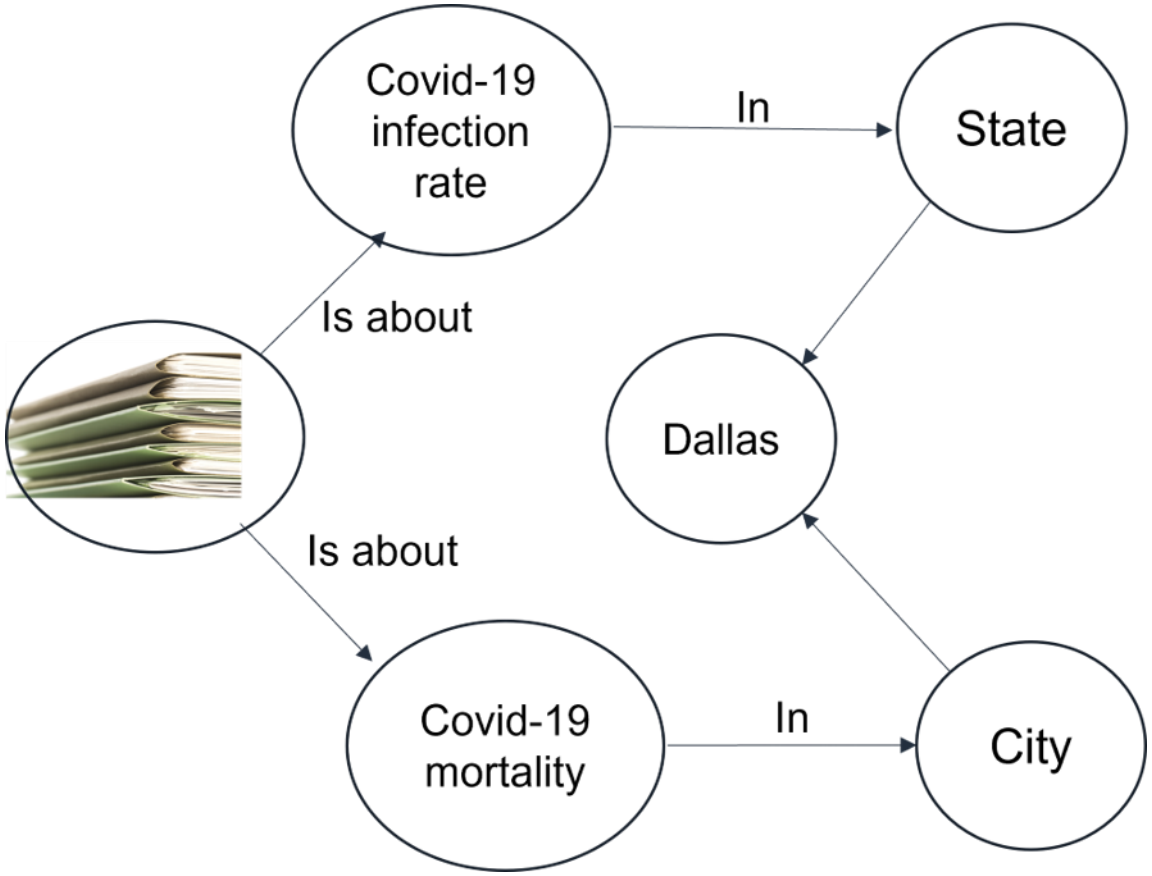
- S3 buckets are used to store ML artifacts
  - Raw and processed datasets
  - Models
- Amazon S3 is a commonly used storage service for ML
- Amazon S3 is used for NLP problems that do not require high-scale searches or learning knowledge searches

Amazon ES use cases

- Build visualizations that are based on complex searches of text data
- Access time-series views of text
  - Track customer reviews over time
- View relationships between documents in a collection
  - Word clouds of common terms in a document set
- Perform complex search integration with NLP interfaces
  - Searchbot for documents or images

Amazon Neptune use cases

- Navigation tools for
  - Discovery of scientific research papers
  - Recommendation engines
- Visual representation of data
  - Employment and occupation and trends for human resource planning
  - Detecting fraudulent transactions
  - Network traffic analysis and planning
- Maps of geographic information
  - Global operations information
  - Shopping applications that are based on GPS data



Section 6 summary

- Structured data –
  - Conforms to a specific schema
    - Predetermined set of fields
    - Set data types for each field
  - Is supported by traditional DBMS
    - Queried by Structured Query Language (SQL)
- Unstructured data –
  - Variation in fields and data types

- Amazon S3
  - Virtually unlimited object storage
  - Most common datastore for ML
  - Suitable for both structured and unstructured data
- Amazon ES
  - Managed service for building search applications
- Amazon Neptune
  - Managed service for building graph database applications

Thank you

Corrections, feedback, or other questions?
Contact us at https://support.aws.amazon.com/#/contacts/aws-academy.